

Wire-Level Interrupt-to-Decision Latency of On-Sensor MLC versus Host Inference on the NVIDIA Jetson Orin Nano: A Pre-Registered Measurement Study

Akul Swami¹ and Dnyaneshwar Sonawane²

¹*Independent Researcher (e-mail: swami.akul@alumni.uml.edu)*

²*Independent Researcher (e-mail: sonawane.dnyaneshwar19@gmail.com)*

Manuscript submitted May 28, 2026.

Abstract—The Machine Learning Core (MLC) embedded in the STMicroelectronics LSM6DSOX IMU is widely cited as a low-latency alternative to host-side inference, yet wire-level decision-delivery latency is rarely measured. Using a Saleae Logic Pro 8 logic analyzer on an NVIDIA Jetson Orin Nano, we measured interrupt-to-decision latency (sensor INT1 edge to host decision GPIO) for three pipelines (a host-side decision-tree classifier, the standard MLC bank-switch read protocol, and an MLC binary-fast variant) under idle, I²C bus contention, and CPU stress. The protocol was pre-registered with 13 externally-timestamped Zenodo amendments (4,770 of 4,860 trials included, 98.15%, across nine cells). The host pipeline exhibits lower median latency than the MLC pipeline under all conditions: 321.7 vs 681.5 μ s at idle (2.1 $\times$ faster) and 574.5 vs 1,325.4 μ s under I²C contention (2.3 $\times$ faster). The three-transaction I²C read protocol, not the silicon's classification, is the dominant latency contributor. We additionally characterize a reproducible 706.5 ms MLC decision cadence that bounds full stimulus-to-decision latency. Code, data, and pre-registration: github.com/akulswami/sensor-mlc-latency.

Index Terms—Inertial measurement unit, machine learning core, edge inference, latency measurement, pre-registration, LSM6DSOX.

I. INTRODUCTION

Inertial measurement units (IMUs) with embedded Machine Learning Cores (MLCs) are increasingly deployed in edge applications spanning wearables, industrial monitoring, and biomedical instrumentation [1]. The MLC paradigm runs a decision-tree classifier inside the sensor and exposes only an output register to the host. It is often framed as a latency and energy advantage over host-side classification [2], [3], on the implicit assumption that on-sensor inference is unconditionally lower-latency because the host need only poll a single register.

This assumption is rarely tested with wire-level instrumentation. Vendor application notes characterize MLC architecture and per-class accuracy but not end-to-end stimulus-to-decision latency [2]. Even recent safety-critical applications such as exoskeleton control [3] cite the on-sensor latency advantage without measuring it directly. The resulting gap between “the classification is correct” and “the classification reaches the host in time to act” is the failure mode conventional functional testing misses. Framed in sensor-system terms, the question is whether a commercial on-sensor inference engine can deliver its decision to the host within a latency budget—a deployment-capability property of the sensor device itself, dictated by how it exposes its on-chip inference output, not by any host-side algorithm.

We measure wire-level interrupt-to-decision latency (sensor INT1 edge to host decision GPIO) for three pipelines on a representative edge platform (NVIDIA Jetson Orin Nano + STMicroelectronics LSM6DSOX over I²C) using a Saleae Logic Pro 8 logic analyzer: (a) a host-side decision-tree classifier reading raw accelerometer data, (b) the standard MLC bank-switch read protocol, and (c) an MLC binary-fast variant that omits the I²C read entirely, toggling the decision

GPIO on every MLC interrupt (valid for a 2-class configuration). Measurements span idle, I²C bus contention, and CPU saturation. The full protocol was pre-registered with 13 externally-timestamped Zenodo amendments [4].

The principal finding is that the host pipeline exhibits lower median wire-level latency than the standard MLC bank-switch pipeline under all tested conditions, with the three-transaction I²C read protocol, not the silicon's classification, as the dominant latency contributor. Thus, the contribution is a quantitative characterization of a sensor-device output path—INT1 assertion, embedded-bank readout, and host-visible decision delivery—not a new AI/ML classifier. Our contributions are: (1) the first pre-registered wire-level latency characterization of the LSM6DSOX MLC versus host-side inference (4,770 of 4,860 trials included, 98.15%, across nine pipeline $\times$ condition cells); (2) empirical identification of the LSM6DSOX MLC's mandatory embedded-bank read protocol—not its classifier—as the dominant latency contributor, a property intrinsic to the sensor's output architecture; (3) characterization of a reproducible 706.5 ms MLC decision cadence (the 75-sample $\times$ 104 Hz window period) that bounds full stimulus-to-decision latency; and (4) pre-registration with externally-timestamped Zenodo DOIs as an audit-defensible framework for sensor-measurement claims.

II. BACKGROUND AND RELATED WORK

A. On-sensor inference in MEMS IMUs

Embedded machine learning in 6-axis microelectromechanical systems (MEMS) IMUs is now a standard product feature. STMicroelectronics introduced the MLC in the LSM6DSOX [2], a configurable decision-tree engine running at the accelerometer ODR (104 Hz in our deployment) that consumes features over sliding sample windows and emits class labels the host reads via I²C bank-switched registers. Successor parts (LSM6DSV16X, LSM6DSO32X) retain this architecture, and competing vendors offer analogues (Bosch

Corresponding author: A. Swami (e-mail: swami.akul@alumni.uml.edu).

Code and data: <https://github.com/akulswami/sensor-mlc-latency>.

The authors declare no conflict of interest.

BHI260AP, Analog Devices ADXL367). The consistent commercial messaging is that on-sensor inference reduces data-bus traffic, host CPU utilization, and end-to-end latency. The first two claims are testable from system-level metrics; the third is testable only with wire-level instrumentation, which is rarely performed.

B. Wire-level latency in safety-critical edge ML

Safety-critical edge applications require bounded stimulus-to-actuation latency, not merely correct classification. Existing MLC documentation emphasizes architecture and accuracy [2], while independent characterizations rarely isolate the sensor read protocol's contribution to decision-delivery delay.

C. Pre-registration in sensor measurement

Pre-registration is used here to separate pre-specified sensor-measurement hypotheses from post-hoc interpretation.

III. EXPERIMENTAL SETUP

A. Hardware Platform

The edge platform is an NVIDIA Jetson Orin Nano Developer Kit (JetPack 6.2.2, kernel 5.15.148-tegra, 6-core ARM Cortex-A78AE). All measurements ran under a custom MAXN_SUPER_JC nvmodel (v7.6 [4]) pinning all six CPUs at 1728 MHz; per-block jetson_clocks effectiveness (tegrastats samples ≥ 1700 MHz) was $\geq 99\%$, with all 81 blocks at 100%.

The sensor is an STMicroelectronics LSM6DSOX 6-axis IMU on an Adafruit breakout (PID 4438; onboard 3.3 V regulation and 10K I²C pull-ups) on I²C bus 7 (pins 3/5, address 0x6A), the bus running at 400 kHz (Fast Mode, Jetson default). All three pipelines configure the accelerometer identically (CTRL1_XL = 0x50: 208 Hz, ± 2 g, LPF2 off). The MLC runs at 104 Hz (the configured accelerometer/MLC output data rate, distinct from the gyroscope_odr field reported as 26 Hz in the ST MEMS Studio export) with a custom 2-class motion/still classifier (75-sample windows, config mlc_motion_w75.h) trained in ST MEMS Studio, with output routed to INT1.

Two Jetson GPIO lines are instrumented with a Saleae Logic Pro 8 (250 MS/s): **D0** on pin 15 (INT1 from the LSM6DSOX, gpiochip0 line 85) and **D1** on pin 11 (decision edge from the pipeline under test, line 112). This yields nanosecond-resolution wire-level timestamps independent of the Jetson clock and of SSH command jitter (both D0 and D1 are wire edges). The servo PWM command is additionally captured on D2 as a timing reference. The wire-level GPIO timestamping methodology follows our prior cross-platform characterization [9]. Motion stimulus is an SG90 servo on a PCA9685 PWM controller (I²C bus 1, address 0x60; the board default 0x40 collides with the on-board INA3221, so an address-select pad was bridged to set the A5 bit, relocating the controller to 0x60), commanded over SSH. The full measurement setup is shown in Fig. 2.

B. Pipelines Under Test

Three pipelines are compared end-to-end, sharing the same I²C arbitration, gpiod write path, and accelerometer configuration, and differing only in what happens after an interrupt arrives:

(a) **host** (host_pipeline_parity): polls the accelerometer at 208 Hz, assembles a 75-sample window, and applies a decision-tree classifier (variance of accelerometer L2-norm against a calibrated threshold), toggling D1 on every output state change.

Host window and MLC window are both 721.2 ms: parity_core's pc_step implements a 2:1 decimation of the host sample stream, equalizing the two windows to a common period.

(b) **mlc** (latency_test_mlc_w75): on each INT1 edge, performs the three I²C transactions of the bank-switch read mandated by the LSM6DSOX register architecture [2] (write FUNC_CFG_ACCESS = 0x80 to enter the embedded-function bank, read MLC0_SRC = 0x70, write FUNC_CFG_ACCESS = 0x00 to restore the user bank), writing D1 if the output changed.

(c) **mlc-binary** (latency_test_mlc_binary_w75): on each INT1 edge, unconditionally toggles D1 without reading MLC0_SRC, valid for the 2-class case and providing a kernel/gpiod-only latency floor against which (b)'s I²C-read overhead is measured. Because this study measures decision-delivery latency rather than classification accuracy, parity is enforced at the level of identical accelerometer configuration, window length, and binary motion/still decision semantics; the mlc-binary condition isolates the non-classifier read path.

The host pipeline sets INT1_CTRL = 0x01 (DRDY); mlc and mlc-binary set INT1_CTRL = 0x00, routing the MLC's decision via MD1_CFG = 0x02 instead — intrinsic to each pipeline's decision source (raw sample vs. embedded classifier).

C. Stress Conditions

Three conditions are tested. **idle**: CPUs pinned but otherwise unloaded. **i2c-contention**: three concurrent i2c_hammer processes continuously read non-MLC registers from the LSM6DSOX on bus 7, contending for bus access. **stress**: stress-ng 0.13.12 saturates all six CPUs (–cpu 6 –cpu-method matrixprod). Hammer-process count and CPU saturation are verified at block start.

D. Measurement Protocol

Each (pipeline, condition) cell comprises 9 blocks of 300 s. Block order across the 81-block campaign follows a pre-registered deterministic shuffle (seed 1990185399 [4]). Within each block the orchestrator delivers 60 candidate transitions; latency is $t(D1) - t(D0)$. Inclusion (pre-registered v7.4) requires exactly one D1 rising edge, preceded by an unambiguous single D0 edge, per stimulus window; violations are categorized and reported as a classifier-stability outcome (Section V.D). Across the campaign, 4,770 of 4,860 trials (98.15%) were included, no cell exceeding the pre-registered 10% exclusion ceiling.

IV. METHODOLOGY AND STATISTICAL TREATMENT

A. Pre-registration

Hypotheses, exclusions, statistical tests, and multiplicity correction were specified in a public pre-registration chain with Zenodo DOIs [4]. The chain records substantive hypothesis and methodology changes before the affected confirmatory data. Three falsified hypotheses remain on the public record: pilot-stage H1 and H4', and confirmatory H7'. Thus, the record distinguishes pre-specified predictions from post-hoc interpretation.

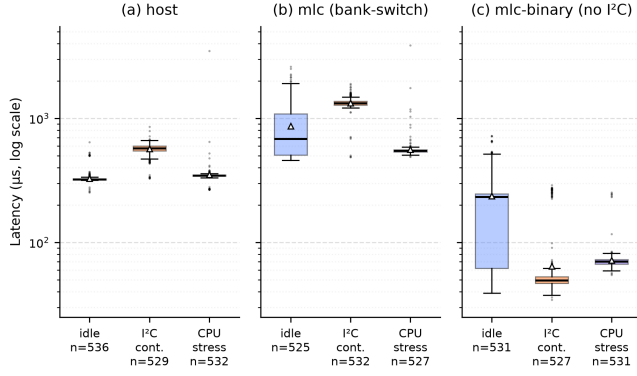


Fig. 1. Per-trial wire-level latency by pipeline and condition, on one shared log-scale y -axis across all three panels so box position is directly comparable across pipelines. The mlc-binary panel (zero I²C transactions on the decision path) isolates the kernel/gpio latency floor. Box = IQR (Q1–Q3); whiskers = $1.5 \times \text{IQR}$; Δ = mean; $\bullet$ = outlier; every point is shown in-frame, with no per-panel capping.

B. Statistical tests and multiplicity correction

Each hypothesis is matched to a pre-registered test. H1' and H2' (stochastic ordering of two independent samples) use a one-sided Mann-Whitney U test, with the Hodges-Lehmann shift and a 95% percentile-bootstrap CI (10,000 resamples) as the effect estimate [5], [6]. H3' (the interaction contrast $\Delta_{\text{MLC}} - \Delta_{\text{host}} > 0$) bootstraps the contrast statistic (10,000 resamples). H5' (equivalence within $\pm 30 \mu\text{s}$) uses TOST [7]: the 90% CI on the median difference must lie strictly within $[-30, +30] \mu\text{s}$. H6' (energy difference $> 1,000 \text{ mW}$) uses Mann-Whitney U on VDD_IN samples with 1/10 subsampling to reduce the autocorrelation of 100 ms-cadence INA3221 readings. H7' (stability ordering) uses a one-sided Fisher's exact test, chosen over chi-square because the per-cell unstable counts (6–15 of 540) fall where the chi-square asymptotic approximation is unreliable.

Multiplicity correction across the H1', H2', H3', H6', H7' family is by Holm-Bonferroni at family-wise $\alpha = 0.05$ [8]. H5' uses TOST equivalence rather than a directional p-value and is reported alongside the family rather than within it; this is an analysis-side refinement of the v7.5 specification, which had named a six-hypothesis Holm family.

V. RESULTS

The confirmatory campaign collected 4,860 candidate trials across 81 blocks of 300 s; 4,770 (98.15%) satisfied the pre-registered inclusion criteria. All blocks achieved 100% jetson_clocks effectiveness under MAXN_SUPER_JC. Per-cell latency distributions appear in Fig. 1; summary statistics in Table I.

A. Confirmatory tests

H1' (host < MLC at idle): SUPPORTED. Host idle median ($321.7 \mu\text{s}$) is $359 \mu\text{s}$ below MLC bank-switch idle ($681.5 \mu\text{s}$), a $2.1 \times$ host speedup. The Hodges-Lehmann shift is $\Delta_{\text{HL}} = -359.3 \mu\text{s}$, 95% bootstrap CI $[-373.6, -352.5]$ (10,000 iterations); Mann-Whitney $p = 1.87 \times 10^{-170}$, rejected at the Holm-Bonferroni threshold across the H1', H2', H3', H6', H7' family.

H2' (host < MLC under I²C contention): SUPPORTED. With three concurrent i2c_hammer processes on bus 7, host median rises

TABLE I. Wire-Level Latency per Pipeline $\times$ Condition (μs)

Pipeline	Condition	n	Median (μs)	IQR (μs , p25–p75)
host	idle	536	321.7	319.7–326.3
host	i2c-cont.	529	574.5	547.8–599.0
host	stress	532	345.0	342.0–348.9
mlc	idle	525	681.5	505.4–1086.8
mlc	i2c-cont.	532	1325.4	1283.7–1370.8
mlc	stress	527	546.1	535.9–557.0
mlc-binary	idle	531	231.9	61.8–246.2
mlc-binary	i2c-cont.	527	49.4	46.9–53.1
mlc-binary	stress	531	70.2	66.9–73.0

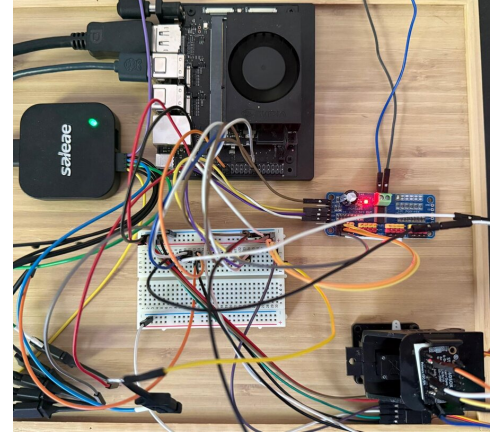


Fig. 2. Measurement setup. The LSM6DSOX IMU (on a servo-driven motion stimulus) and the PCA9685 servo controller connect to the NVIDIA Jetson Orin Nano over separate I²C buses; a Saleae Logic Pro 8 taps the sensor INT1 edge (D0), the host decision GPIO (D1), and the servo PWM command (D2) for wire-level timing.

to $574.5 \mu\text{s}$ and MLC to $1,325.4 \mu\text{s}$; the shift grows to $\Delta_{\text{HL}} = -753.2 \mu\text{s}$ $[-760.0, -746.7]$ ($2.3 \times$ advantage), $p = 2.33 \times 10^{-170}$.

H3' (MLC degrades more than host): SUPPORTED. The Δ -of- Δ contrast is $+391.1 \mu\text{s}$ $[+372.8, +400.0]$, $p = 1.00 \times 10^{-4}$. The MLC degrades by $+611.7 \mu\text{s}$ ($+90\%$, idle $\rightarrow$ contention) versus the host's $+249.2 \mu\text{s}$ ($+78\%$): the MLC's three-transaction I²C read protocol amplifies the contention penalty.

H5' (CPU stress null for host latency): SUPPORTED. Host median rises only from $321.7 \mu\text{s}$ (idle) to $345.0 \mu\text{s}$ (stress); a TOST against the pre-registered $\pm 30 \mu\text{s}$ margin gives $+23.3 \mu\text{s}$, 90% CI $[+22.7, +23.7] \subset [-30, +30]$. Equivalence is declared; a 208 Hz polling loop does not contend with stress-ng for CPU time.

H6' (CPU stress positive for energy): SUPPORTED. Mean VDD_IN (INA3221 via tegrastats) rises from $5,206 \text{ mW}$ (idle) to $8,626 \text{ mW}$ (stress): $+3,420 \text{ mW}$ $[+3,410, +3,429]$, exceeding the pre-registered $+1,000 \text{ mW}$ threshold threefold; H6' is a manipulation check on the CPU-stress condition, not a host-versus-MLC energy comparison (see supplementary S2 for that post-hoc comparison). The power axis distinguishes CPU stress unambiguously where the latency axis (H5') cannot.

H7' (MLC stability degrades under contention): FALSIFIED, direction opposite. The fraction of stimulus windows with exactly one D1 rising edge is 97.22% (525/540) at idle versus 98.89% (534/540) under contention, a $+1.67$ percentage-point *increase* rather than the predicted decrease. Fisher's exact test in the pre-registered direction gives $p = 0.9874$ (two-sided $p = 0.0755$). H7' is formally falsified

in pre-registration v7.10 [4]. I^2C contention slows the MLC pipeline (H2', H3') but does not degrade the silicon's classifier reliability.

B. Multimodal latency distributions

Fig. 1 reveals multimodal structure in both MLC pipelines: the mlc/idle distribution is bimodal (mean 866.6 μs exceeds median 681.5 μs , p95 1,779.7 μs), and mlc-binary/idle shows three modes near 60, 240, and 470 μs . The mlc-binary/i2c-contention distribution has a hidden second mode: median latency is 49.4 μs , but p95 reaches 246.7 μs , indicating a secondary high-latency population not captured by the median alone. Both collapse to tighter distributions under contention and stress. We report this as exploratory.

C. MLC decision cadence

Inter-trial D0 (INT1) gaps ($n = 3,086$) cluster sharply at integer multiples of $T = 706.5$ ms, approximately the MLC's 75-sample, 104 Hz window period (0.721 s expected). The quantization is consistent with the MLC updating only on its internal window-cadence clock.

D. Exclusion rates

No cell exceeded the pre-registered 10% exclusion ceiling (highest: mlc/idle, 2.78%). The dominant exclusion category (68 of 90 trials) was multiple D1 edges per window, attributable to the 706.5 ms cadence interacting with stimulus-window boundaries. In the MLC/ I^2C -contention cell, inclusion ($n=532$, Table I) and the H7' stability count ($n=534$, Section V.A) use related but distinct single-edge criteria: inclusion additionally requires an unambiguous single D0 rising edge before the single D1 edge (Section III.D), which the H7' criterion does not.

VI. DISCUSSION

A. The I^2C read protocol dominates wire-level latency

The headline finding, that host classification is 2.1–2.3 $\times$ faster than the on-sensor MLC at idle and under I^2C contention, is explained by **Fig. 1(c)**: the mlc-binary pipeline, which performs zero I^2C transactions on the decision path (toggling D1 unconditionally on every INT1 edge), reaches a median of 49.4 μs under contention. The mlc-minus-mlc-binary difference under identical conditions isolates the I^2C read overhead at roughly 1,276 μs under contention (1,325.4 – 49.4) and 476 μs under stress. Under CPU stress, where the I^2C bus itself is uncontended, the host-MLC gap narrows to 1.6 $\times$ (345.0 vs 546.1 μs): the read protocol still costs 476 μs , but without concurrent bus traffic to amplify it. This cost is not the silicon's classification time; it is the bank-switch read protocol's three-transaction sequence (write FUNC_CFG_ACCESS, read MLC0_SRC, write it back) competing for bus arbitration.

B. Implications for safety-critical edge ML

A naive reading of “on-sensor inference is faster” would favor the MLC for low-latency safety-critical loops such as exoskeleton control [3]. Our results invert that on the wire-level latency axis: the host reaches its decision 359 μs earlier at idle, 753 μs earlier under contention, and is equivalence-null against CPU stress (H5').

The control results sharpen the practical lesson: **“stress” is not a single thing**. CPU stress is significant on energy but not latency; I^2C contention is significant on latency but not classifier reliability. Bundling both into one “stress margin” mis-budgets for each.

C. Multimodal distributions and decision cadence

Two secondary findings carry safety-critical weight. First, the MLC pipelines are multimodal at idle (Section V.B): the mlc/idle p95 of 1,780 μs is 2.6 $\times$ its median, a factor that vanishes under a unimodal-Gaussian assumption. For a “worst latency observed with probability $1 - \epsilon$ ” specification, the upper mode, not the median, is the relevant quantity.

Second, for unsynchronized external stimuli, the observed 706.5 ms cadence (Section V.C) can dominate full *stimulus-to-decision* latency at the system level. Because the MLC fires only on its internal clock boundary, an unsynchronized real-world stimulus waits a uniformly-distributed 0–706.5 ms (mean 353 ms) before the silicon can respond. This is invisible on the D0-to-D1 wire-level axis we measured but is a structural floor; the 1–2 ms wire-level differences this paper characterizes are second-order against it.

D. Limitations

Results are specific to one platform, sensor IC family, bus protocol (I^2C , not SPI), ODR, and MLC configuration; the structural findings should transfer to similar ARM-edge + ST-MEMS combinations but require confirmation. SPI access in particular could reduce the per-transaction overhead that drives our result. The three-transaction bank-switch cost reflects this implementation's choice to restore the user bank after every MLC0_SRC read, not an unconditional silicon requirement: an application that caches bank state and defers the switch-back could avoid at least one transaction (not measured here). SPI access would reduce this per-transaction overhead but was not measured, so no speedup is quantified. Successor LSM6DSOX-family parts (LSM6DSV16X, LSM6DSO32X) retain the same bank-switched architecture, so the overhead is not specific to this part.

A pre-registered RT-scheduling (chrt+taskset) ablation was specified but not activated; pilot data suggest it could roughly halve MLC contention latency, making it the most concrete next step.

VII. CONCLUSION

We measured wire-level interrupt-to-decision latency for three pipelines on a Jetson Orin Nano + LSM6DSOX edge platform. The host pipeline is consistently faster; the three-transaction I^2C read, not the silicon's classification, dominates.

REFERENCES

- [1] STMicroelectronics, “LSM6DSOX: 6-axis IMU with machine learning core,” Datasheet DS12140, 2024.
- [2] STMicroelectronics, “LSM6DSOX: Machine Learning Core,” Application Note AN5259, 2023.
- [3] M. Razmi and I. Shojaei, “Event-driven on-sensor locomotion mode recognition for exoskeleton control,” arXiv:2602.21418, Feb. 2026.
- [4] A. Swami, “Pre-registration chain for the sensor-mlc-latency study (v6.1–v7.13),” Zenodo, 2026, doi:10.5281/zenodo.22907600.
- [5] H. B. Mann and D. R. Whitney, “On a test of stochastic ordering of two random variables,” *Ann. Math. Statist.*, vol. 18, pp. 50–60, 1947.
- [6] J. L. Hodges and E. L. Lehmann, “Estimates of location based on rank tests,” *Ann. Math. Statist.*, vol. 34, pp. 598–611, 1963.
- [7] D. J. Schuurman, “Two one-sided tests procedure for assessing equivalence,” *J. Pharmacokinet. Biopharm.*, vol. 15, pp. 657–680, 1987.
- [8] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.
- [9] A. Swami and N. Chougule, “Per-platform GPIO overhead in hardware-validated edge ML inference timing,” arXiv:2605.02835, May 2026.